

RockMoss Shader Documentation

A URP Lit shader designed to seamlessly blend a base texture with a procedural top cover layer (e.g., moss, snow). It utilizes dual Triplanar projection, completely eliminating the need for manual UV unwrapping.

Properties

- **RockBC:** The primary texture (Base Map) for the base material.
- **BaseColor:** Additional base tint applied to the main texture.
- **TextureTiling:** The repetition scale for the base RockBC texture.
- **Smoothness:** The surface smoothness value. In low-poly styling, this is typically kept near zero to reduce unwanted specular highlights on flat polygon faces.
- **CoverBC:** The texture used for the procedural top cover layer.
- **CoverTint:** Additional color tint multiplied over the CoverBC texture.
- **CoverTextureTiling:** The repetition scale for the top cover texture.
- **CoverStr:** Defines the cutoff threshold and coverage strength of the procedural layer based on the surface slope angle of the model. **Model Preparation**
- **Triplanar Mapping:** Procedural projection using Triplanar nodes means the model's standard UV mapping is completely ignored when rendering textures. This is a massive workflow enhancement for the mass production of raw, blocky meshes, allowing entire sets of objects to be deployed into the scene instantly without pausing for UV unwrapping.
- **No Vertex Color Support:** The shader fully automates masking by reading the surface normals. Unlike other materials in this package, it does not use Vertex Painting to define layer boundaries.

SG_Foliage Shader

It features an integrated procedural wind system driven by Vertex Animation. The shader calculates two distinct levels of wind: macro-movement for the entire tree/trunk bending, and micro-movement for individual leaf fluttering. It also includes a debug toggle to visualize vertex colors directly on the mesh.

Properties

- **Base Material:** The main Base Map texture covering both the trunk and the foliage.
- **LeafColourTint:** Additional color tint applied specifically to the foliage areas.
- **Alpha Clip Threshold:** Cutoff value for the leaf transparency (Cutout).
- **Emissive Mask:** Texture mask defining the glowing areas of the mesh.
- **Emissive Color:** Color and intensity multiplier for the emissive mask.
- **Wind Speed / Wind Density / Wind Strength:** Parameters controlling the slow, large-scale macro-movement (trunk bending).
- **Small Wind Speed / Small Wind Density / Small Wind Strength:** Parameters controlling the fast, high-frequency micro-movement (leaf fluttering).
- **ShowVertexColor:** A debugging toggle. When enabled, it replaces the material's base color with the raw vertex colors of the mesh, allowing you to easily verify your painted weights.

Model Preparation (Vertex Painting) The procedural wind deformation relies entirely on Vertex Colors painted onto the mesh. To ensure the wind works correctly, you must configure the vertex weights as follows:

- **Black (0, 0, 0):** No movement (Anchor). Use this exclusively at the very bottom of the trunk and roots to keep the tree firmly planted on the ground.
- **White (1, 1, 1):** Full macro and micro movement. Use this at the outer tips of the canopy for maximum wind influence.
- **Blue Channel (B):** Macro movement (Trunk bending / Global wind). Controls how much the entire tree sways. Create a smooth gradient from 0 at the base to 1 at the top of the tree.
- **Red Channel (R):** Micro movement (Leaf fluttering). Controls the rapid fluttering of the leaves. Isolate the foliage with this channel so the leaves can flutter independently without distorting the solid trunk.
- **Green Channel (G):** Currently unused by this shader.

SG_Lit Shader

A URP Lit shader extended with a procedural top cover layer system and support for texture-based emission.

Properties

- **BaseMap:** The primary texture of the object.
- **Smoothness:** Adjusts surface smoothness and the clarity of light reflections.
- **useTextureAsEmissive:** Enables treating the main texture as a map defining glowing areas.
- **EmissionColor:** Sets the base color and overall emission power.
- **EmissionStrength:** Modifier for the intensity of the material's glow.
- **useCoverLayer:** Main toggle that activates the top cover layer system.
- **CoverBC:** Texture used for the Cover layer.
- **CoverTint:** Additional tint applied to the Cover texture.
- **CoverTextureTiling:** Determines the tiling scale of the Cover texture.
- **CoverStr:** Controls the effect's strength and coverage extent based on the surface slope angle.

Model Preparation

- **Vertex Painting:** The shader utilizes the Red channel of the Vertex Color strictly as an Emission mask. Painting a vertex black (R=0) completely disables emission in that area, while painting it white (R=1) allows the full emission effect to pass through. This allows for precise, hand-painted control over which parts of the mesh should glow without needing a separate emission texture map. Please note that vertex color does not mask or affect the procedural Cover layer.
- **UV Mapping:** The Cover layer is applied entirely procedurally (triplanar) and ignores the object's UV map. However, proper UV mapping is still required to correctly display the primary BaseMap texture.

VertexColor Shader

A highly optimized URP Lit shader that completely bypasses standard texturing. Instead, it reads the mesh's Vertex Colors and uses them as masks to smoothly blend three independent, fully customizable color properties. **Properties**

- **VertexColorRed:** The color applied to the areas of the mesh painted red (R channel value in Vertex Color).
- **VertexColorGreen:** The color applied to the green vertex areas (G channel value in Vertex Color).
- **VertexColorBlue:** The color applied to the blue vertex areas (B channel value in Vertex Color).

Model Preparation (Vertex Painting) Since this shader relies entirely on geometry data, the mesh requires specific preparation before exporting from your 3D software:

No UV Mapping Required: Coloring objects with this shader completely frees you from the obligation of UV unwrapping for albedo maps.

Vertex Painting: To assign the colors defined in the material to specific polygons, paint the model using pure RGB values (Red: 1,0,0; Green: 0,1,0; Blue: 0,0,1). Blending these colors at the vertices will result in smooth transitions between the defined properties.

Water Shader

Shader designed specifically for water surfaces and aquatic environments. It features a modular effect structure that includes depth-based color absorption, procedural edge foam, dynamic waves, and fine surface ripples.

Properties

General:

- **WaterColor:** The primary, base color of the water surface.
- **Scale:** A global multiplier that scales the procedural noise patterns generated on the surface.
- **Speed:** A global time multiplier that affects the overall speed of all animated elements on the surface.

Depth & Absorption:

- **AbsorptionColor:** The color the water smoothly transitions into at greater depths.
- **AbsorptionCoefficient:** A coefficient controlling how abruptly the water changes color from shallow to deep, based on the scene's depth buffer reading.
- **Depth:** A parameter modifying the strength of depth detection for underwater visual effects.

Foam:

- **FoamColor:** The color of the procedural foam generated where the water mesh intersects with other solid objects (e.g., shores, rocks, terrain).
- **FoamAmount:** Determines the overall spread and intensity of the generated foam.
- **FoatCutOff:** (Intended as *Foam CutOff*) A clipping threshold that removes fainter foam values, resulting in harder, sharper foam edges.

Wave:

- **WaveScale:** Adjusts the size and frequency of the large waves on the water.
- **WaveStrength:** Defines the amplitude of the waves (the strength of the vertex displacement/mesh distortion).
- **WaveSpeed:** Controls the movement speed of the large waves across the surface.

Ripple:

- **RippleDensity:** Determines the concentration and scale of the fine ripples on the water (e.g., wind disturbance).

- **RippleSlimness:** Affects the thickness of the procedural ripples, allowing for sharper and narrower lines across the water.
- **RippleMultiply:** An intensity multiplier that enhances the visibility of the fine ripple effect.
- **RippleSpeed:** The speed at which the ripples travel across the surface, operating independently of the main waves.

Additional Effects:

- **WaterfallSpeed:** A parameter that independently controls the scrolling speed of the effects along an axis (usually Y). This is highly useful when assigning the material to vertical waterfall meshes.

Model Preparation

- **Mesh Subdivision:** For wave distortion to function smoothly (as it relies on vertex displacement), the water mesh must be adequately subdivided into smaller polygons. A simple 4-vertex quad will not bend to form a wave shape.
- **Depth Texture Requirement:** The edge foam and color absorption effects rely entirely on reading the scene's depth. Ensure that the **Depth Texture** option is enabled in your project's URP Asset settings, and that the objects intersecting the water are rendered as Opaque.